Direct closure induction for iPRA

An equivalent primitive presentation and its measured limits

Research package prepared for Vladimir Reshetnikov

November 10, 2026

Abstract

The current ConciseIPRA calculus introduces equation induction inside an existentially witnessed paired binder. This note removes that witness from the primitive induction rule: from $\Gamma, E(z) \vdash E(Sz)$, infer $\Gamma \vdash Q_y E(y)$, for a literal equation E. The paired binder keeps only its case introduction. Explicit finite proof transformations show that the new and old calculi prove exactly the same sequents, with arbitrary native assumptions. Hence the existing whole-domain bi-interpretation with full first-order intuitionistic primitive recursive arithmetic transfers unchanged.

The result is a modest refactoring of the primitive interface. The admissible rule already occurs as `NPrf.qIProg` in the pinned Lean development. It is not a new induction principle or a new arithmetic completeness theorem. Most importantly, it does not improve the repository's complete-checker size record: the two implementations here measure 568 and 552 normalized Python bytes, against 529 for the current byte profile. Thus the stricter request for a smaller complete proof checker remains unresolved. This distinction is part of the result, not a qualification deferred to the end.

Established: the induction rule has no existential witness; every native sequent is preserved; full bi-interpretability and equiconsistency transfer; proof translations and representative certificates are executable.
Not established: a smaller complete checker than the current frontier, an absolutely simplest axiomatization, or a new Lean/Rocq verification of the replacement calculus.

Contents

1 Comparison point and the meaning of simplicity **3**

2 A complete primitive presentation **3**

2.1 Terms, formulas, and contexts 3

2.2 All axioms and inference rules 4

2.3 The single change from the current calculus 5

3 Exact preservation of every native sequent **5**

3.1 An optional deterministic closure rule 6

4 Full iPRA, both interpretations, and both composites **7**

4.1 The ordinary arithmetic realization 7

4.2 Why ordinary intuitionistic logic is retained 7

4.3 The inherited scalar arithmetic bridge 8

5 A concrete proof that becomes shorter **9**

6 Implementation comparison and validation **10**

6.1 The complete-source measurement 10

6.2 Certificate interface 10

6.3 What was checked 11

7 Why several more tempting deletions were not adopted **12**

8 Further research questions **12**

A Complete replacement checker **13**

B Reproduction and provenance **13**

1 Comparison point and the meaning of simplicity

The comparison is with Vladimir Reshetnikov's ConciseIPRA project at the fixed commit

`8662bc0a00f4e77e482f7d141d7a4f919f70f9aa`.

Its path is `Logic/PrimitiveRecursiveArithmetic/ConciseIPRA`. The project README identifies the native scalar core and records the complete-checker metrics of its research packages [1]. The present snapshot was retrieved on November 10 in the user's Pacific time zone (November 11 UTC).

Two notions of simplicity must be distinguished here. A primitive inference can require less mathematical data even when a particular encoding of a complete checker becomes longer. Our replacement removes an existential witness, the substitution of that witness into a two-variable matrix, and its associated freshness condition from the induction rule. It leaves the overall number of primitive inference/axiom patterns at eight. It does not delete a numerical constructor or weaken the equality motive language.

The repository's implementation comparison is stricter. It measures the whole standalone Python module, including certificate input and the public verdict. Its coordinates are significant tokens, Python AST nodes, and UTF-8 bytes after `ast.unparse` normalization; maximum-compression gzip is reported only for information. Under that comparison the present candidates are larger. No source-selection trick or changed acceptance interface is used to manufacture an improvement.

The 529-byte source is the `neutral_scope/code/checker.py` profile. The README also records better token and AST coordinates in other profiles; 529/266/310 is not a claim that one source simultaneously minimizes all three coordinates. Because our candidates already lose against this one concrete complete source, no stronger audit of a coordinatewise frontier is needed to reject a new-record claim.

Attribution. The numerical language, the paired binder, the conditional-universal binder, their ordinary meanings, the logical macros, the scalar arithmetic compiler, and the whole-domain bi-interpretation are inherited. The baseline already derives direct atomic closure progress in `NativeEq.lean` [2]. The positive contribution of this package is the fully stated replacement basis, its two explicit proof transformations, the separation of conceptual and implementation costs, and executable comparative evidence. No worldwide novelty claim is made.

2 A complete primitive presentation

2.1 Terms, formulas, and contexts

There is one numerical sort. Terms are finite expressions

$$t ::= x \mid 0 \mid St \mid I(n, a; u.h).$$

Only u in the update body h is bound. The counter n and initial value a do not lie within that binding. The body is fixed finite syntax and may contain passive numerical parameters. There is no numerical quantification over function descriptions and no uniformly total evaluator for all primitive-recursive programs.

Native formulas are

$$A ::= (s = t) \mid Q_y A \mid D_{x,y} A.$$

The two bound variables of D are distinct. All substitution is capture avoiding, with bound-variable renaming when necessary. A context Γ is a finite reusable list of arbitrary native formulas, including formulas with binders. We write E for a *literal equation*: its root is equality between two numerical terms. A Q or D formula whose innermost expression is an equation is not a literal equation.

The ordinary readings, used to state the interpretation theorem rather than as additional primitive rules, are

$$(s = t)^\circ = (s = t), \tag{1}$$
$$(Q_y B(y))^\circ = B^\circ(0) \to \forall y\, B^\circ(y), \tag{2}$$
$$(D_{x,y} B(x,y))^\circ = \exists x \big(B^\circ(x,0) \to \forall y\, B^\circ(x,y)\big). \tag{3}$$

Thus Q is a conditional universal closure, while D existentially quantifies a witness for that closure.

2.2 All axioms and inference rules

The only computation axioms, available in every retained context, are

$$I(0, a; u.h) = a, \qquad \text{(I0)} \tag{4}$$
$$I(Sn, a; u.h) = h[I(n, a; u.h)/u]. \qquad \text{(IS)} \tag{5}$$

The new calculus $\mathcal{C}_Q$ has exactly the following six inference patterns:

$$\frac{}{\Gamma \vdash A}\ (A \in \Gamma) \qquad \text{(Hyp)}, \tag{6}$$

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash F(s)}{\Gamma \vdash F(t)} \qquad \text{(Tr)}, \tag{7}$$

$$\frac{\Gamma \vdash Q_y B(y) \qquad \Gamma \vdash B(0)}{\Gamma \vdash B(r)} \qquad \text{(QE)}, \tag{8}$$

$$\frac{\Gamma \vdash D_{x,y} B(x,y) \qquad \Gamma, Q_y B(b,y) \vdash A}{\Gamma \vdash A} \qquad \text{(DE)}, \tag{9}$$

$$\frac{\Gamma, B(t,0) \vdash B(t, Sz)}{\Gamma \vdash D_{x,y} B(x,y)} \qquad \text{(DC)}, \tag{10}$$

$$\boxed{\frac{\Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash Q_y E(y)}} \qquad \text{(QP)}. \tag{11}$$

The side conditions are part of these rules:

- In Tr, $F(v)$ is any fixed native formula template. Its numerical placeholder may occur more than once and beneath binders, subject to capture avoidance.
- In DE, b is fresh for Γ, A, and the passive parameters of B.
- In DC, z is fresh for Γ, t, and the passive parameters of B.

- In QP, E is a literal equation, and z is fresh for Γ and the passive parameters of E. There is no existential witness in this rule.

The displayed local assumptions are discharged. These are all the primitive axioms and rules. In particular, there is no primitive reflexivity, successor-injectivity, separation, ordinary logical connective rule, coverage, or further induction schema. The designated native contradiction is $0 = S0$.

2.3 The single change from the current calculus

Call the current calculus $\mathcal{C}_D$. It has the same syntax, computation axioms, Hyp, Tr, QE, DE, and DC, but replaces QP by

$$\frac{\Gamma, E(t,z) \vdash E(t,Sz)}{\Gamma \vdash D_{x,y}E(x,y)} \qquad \text{(DP)}. \tag{12}$$

Here $E(x,y)$ is a literal equation and z is fresh for the context, witness t, and all passive matrix parameters. The change is therefore exactly

$$\text{DP with a witness} \quad \longmapsto \quad \text{QP without a witness.}$$

The existential constructor remains useful for ordinary existential logic; it is removed only from the primitive presentation of equation induction.

3 Exact preservation of every native sequent

Lemma 3.1 (Structural admissibility)**.** *Weakening, exchange, contraction, injective numerical renaming, capture-avoiding numerical substitution, and ordinary-assumption cut are admissible in both $\mathcal{C}_D$ and $\mathcal{C}_Q$.*

Proof. Induct externally on a finite derivation. Before adding a context, term, or proof, rename local eigenparameters away from its free variables. Computation schemas and all the displayed rules are stable under the resulting substitutions. Substitution preserves a literal-equation root, so the induction restriction is preserved. For cut, replace uses of the cut hypothesis with its supplied derivation, weakened to the current local context. Freshening eigenparameters makes each replacement valid. No arithmetic induction is invoked by this induction on proof syntax. □

Theorem 3.2 (Identity-language equivalence)**.** *For every finite native context Γ and every native formula A,*

$$\Gamma \vdash_{\mathcal{C}_D} A \quad \Longleftrightarrow \quad \Gamma \vdash_{\mathcal{C}_Q} A.$$

Both directions are effective finite proof transformations. Terms, contexts, and conclusions are preserved, up to harmless renaming of bound variables.

Proof. Only the two different primitive induction rules require a construction.

Simulate QP in the old calculus. Suppose the already translated premise is

$$\pi : \quad \Gamma, E(z) \vdash E(Sz).$$

Choose a fresh variable x absent from E and Γ. Apply old DP to the matrix $E(y)$, treating x as an unused existential coordinate and using witness 0. This gives

$$\Gamma \vdash D_{x,y}E(y).$$

DE opens this formula at a fresh b. Since x was unused, the opened resource is exactly $Q_yE(y)$, with no occurrence of b. Its continuation is the one-node hypothesis proof

$$\Gamma, Q_yE(y) \vdash Q_yE(y).$$

DE therefore concludes the desired $\Gamma \vdash Q_yE(y)$.

Simulate DP in the new calculus. Suppose the translated old premise is

$$\pi : \quad \Gamma, E(t,z) \vdash E(t,Sz).$$

Form the fixed one-variable equation template $F(y) := E(t,y)$, renaming the bound index first if it occurs freely in t. The original freshness conditions ensure that π is a valid QP premise for F. Thus

$$\Gamma \vdash Q_yE(t,y).$$

To introduce the original paired conclusion, use DC with witness t. Under its local base assumption $E(t,0)$, weaken the closure proof into that context and specialize it by QE at Sw, where w is fresh. The result is

$$\Gamma, E(t,0) \vdash E(t,Sw).$$

DC gives exactly $\Gamma \vdash D_{x,y}E(x,y)$.

Fresh w can also be chosen away from every internal eigenparameter of π. Both constructions use the premise proof exactly once. Replacing these nodes recursively throughout an arbitrary derivation proves the theorem. The simulations use only weakening and fresh renaming: they do not require equality transport or any computation equation. □

Proposition 3.3 (Specified proof-tree costs)**.** *Count each primitive inference, hypothesis, and computation axiom as one node. If a new proof has N nodes and r QP occurrences, its displayed old translation has $N + 2r \le 3N$ nodes. If an old proof has M nodes and s DP occurrences, its displayed new translation has $M + 3s \le 4M$ nodes.*

Proof. A QP node becomes DP, DE, and Hyp: three nodes replace one. A DP node becomes QP, QE, Hyp, and DC: four replace one. Every premise subtree is copied once; weakening and renaming do not add primitive nodes. □

These are bounds on the specified inference trees. They are neither minimum proof lengths nor bounds on JSON bytes or checking time. The two proof translations are not inverse functions on proof objects; their composites preserve the end sequent and may lengthen a proof.

3.1 An optional deterministic closure rule

For implementation comparison it is useful to permit the already admissible Q-case rule on compound matrices. Define $\mathcal{C}_Q^+$ by replacing QP with

$$\frac{\Gamma, B(\eta_B(z)) \vdash B(Sz)}{\Gamma \vdash Q_yB(y)}, \qquad \eta_B(z) = \begin{cases} z, & B \text{ is a literal equation,} \\ 0, & B \text{ has a } Q \text{ or } D \text{ root.} \end{cases} \tag{13}$$

DC is unchanged and always uses its zero case. The mathematical rule is determined by the goal constructor and matrix root; it has no freely supplied case/progress choice for that same goal.

Corollary 3.4. *$\mathcal{C}_D$, $\mathcal{C}_Q$, and $\mathcal{C}_Q^+$ prove exactly the same native sequents.*

Proof. Any Q-case instance is derived by DC with an unused existential variable and witness 0, followed by DE returning the opened Q hypothesis. Thus every new compound case of (13) is already admissible in $\mathcal{C}_Q$. The atomic case of that rule is QP. Conversely every QP instance remains available in $\mathcal{C}_Q^+$. The old/new simulations therefore apply unchanged. □

Although QP alone introduces an atomic Q formula by progress, an atomic Q-case proof is still available through DC and DE. This matters: for example, $Q_y(y = 1)$ is recoverable from its zero-case premise, whose assumption is $0 = 1$. The refactoring does not make that native formula disappear.

4 Full iPRA, both interpretations, and both composites

4.1 The ordinary arithmetic realization

Take ordinary intuitionistic first-order logic with actual equality and genuine falsity. Over the iterator language, let T_{eq} have I0 and IS, coverage, separation, and contextual induction for literal equations:

$$\forall x\,(x = 0 \vee \exists y\, x = Sy), \tag{14}$$

$$(0 = S0) \to \bot, \tag{15}$$

$$\frac{\Gamma \vdash E(0) \qquad \Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash E(t)}, \qquad E \text{ a literal equation.} \tag{16}$$

These describe the *ordinary realization*, not additional native axioms silently added to Section 2. Separation must be stated in this realization: positive numerical equations alone admit a singleton structure, whereas ordinary falsity must be preserved.

Interpret native formulas by (1)–(3), leaving the entire numerical domain and equality unchanged. QP is sound directly: under a temporary assumption $E(0)$, contextual equation induction applied to its premise proves every instance $E(y)$; discharge the base assumption to obtain $E(0) \to \forall y E(y)$. DC is sound by coverage: its retained base handles zero and its premise handles all successors. Hyp, Tr, QE, DE, and both computations have their displayed ordinary interpretations.

The target is full first-order intuitionistic PRA, not just the quantifier-free equational fragment. The usual first-order iPRA literature uses symbols for primitive-recursive functions and atomic induction; quantifier-free induction is recoverable by arithmetic characteristic functions [6]. The pinned project explicitly implements the quantifier-free-induction version and proves the scalar bridge [3].

4.2 Why ordinary intuitionistic logic is retained

The equivalence theorem preserves every native formula and every native context. Consequently all logical macros and all their introduction and elimination rules from the current calculus remain available. This is stronger than an assertion that the two calculi have the same closed equations.

For orientation, the numerical selector is

$$C(v,a,b) := I(v,a;u.b), \qquad C(0,a,b) = a, \quad C(Sn,a,b) = b.$$

The inherited construction lifts selection to native formulas, with $M_0(A,B) \dashv\vdash A$ and $M_{Sn}(A,B) \dashv\vdash B$. Its gate $G_v(A)$ is true at $v = 0$ and equivalent to A at successors. These are finite syntactic constructions; their complete recursive definitions are frozen in `reference/lean/ConciseIPRA/NativeGate.lean`. They are not extra trusted primitives. With

$$p(y) := I(y,0;u.C(u,1,0)), \qquad 1 := S0,$$

the upstream native abbreviations can be written as follows [4]:

Ordinary operation	Native abbreviation
$\bot$	$0 = 1$
$A \to B$	$Q_y M_y(A,B)$
$A \wedge B$	$Q_y G_y(M_{p(y)}(A,B))$
$\exists x\, A(x)$	$D_{x,y} G_y(A(x))$
$A \vee B$	$\exists_* x\, M_x(A,B)$
$\forall y\, A(y)$	$A(0) \wedge_* Q_y A(y)$

Here the stars mean the already displayed native abbreviations, and all new binders are freshened. The baseline derives all intuitionistic natural deduction rules for these operations. Applying the exact proof translation to those derivations recovers the same rules in $\mathcal{C}_Q$.

In particular, the paired-formula round trip is the inherited equivalence

$$D_{x,y}B \;\dashv\vdash\; \exists_* x\big(B(x,0) \to_* \forall_* y B(x,y)\big).$$

Forward, DE exposes a conditional closure, which supplies its instances under the base assumption. Reverse, existential elimination supplies a witness; DC reconstructs the paired formula using the implication and universal at the successor instance. The Q round trip is the corresponding conditional-universal equivalence. External induction on formulas then gives both logical composites. None of these constructions is changed by moving the primitive progress rule.

4.3 The inherited scalar arithmetic bridge

The numerical bridge must be internal, rather than mere standard-model agreement. The project establishes, in order, a scalar predecessor, reflection of actual equality into a numerical characteristic, quantifier-free induction, pairing with its projections, and a compiler for each fixed primitive-recursive definition. The unchanged written details are supplied in `reference/mathematics/`; the frozen Lean development states the final bi-interpretation in `BiInterpretation.lean` [3].

For example, once pairing π and its projections L, H have been proved, the general recursion

$$f(0,\vec{p}) = g(\vec{p}), \qquad f(Sn,\vec{p}) = h(n, f(n,\vec{p}),\vec{p})$$

is compiled by storing the counter with the current value:

$$V(n,\vec{p}) = I(n, \pi(0, g^*(\vec{p})); u.\pi(SL(u), h^*(L(u), H(u), \vec{p}))),$$
$$f^*(n,\vec{p}) = H(V(n,\vec{p})).$$

Equation induction proves $L(V(n,\vec{p})) = n$, and the projection identities then verify the two defining equations. This is an external compilation of each finite description, not a universal evaluator within arithmetic.

In the other direction, iPRA interprets each scalar iterator by a primitive recursion whose update ignores its counter argument. Internal equation induction proves uniqueness of such iterations. External induction on a finite function description therefore identifies each function with its double translation. This is the essential numerical composite argument.

Theorem 4.1 (Bi-interpretability and equiconsistency)**.** *The ordinary realization presented by $\mathcal{C}_Q$ and by $\mathcal{C}_Q^+$ is bi-interpretable and equiconsistent with full first-order iPRA. The inherited interpretations use the entire numerical domain, actual equality, and identity return graph $x = y$ in both theories.*

Proof. The exact-sequent equivalence identifies the new native presentation with the old one on the same language, not merely up to an arithmetic fragment. Compose this identity equivalence with the old native/ordinary translations and with the pinned T_{eq}–iPRA bi-interpretation just described. Every derivation of an interpretation axiom and every derivation of a formula round trip transfers to the new basis. Both numerical domains and operations are unchanged; hence the existing internally provable return isomorphisms still have graph $x = y$. Falsity is preserved through $0 = S0$ and the explicit ordinary separation sentence. Finite proofs of contradiction translate in both directions, giving equiconsistency. □

This proof deliberately reuses the established scalar bridge. It does not claim that the new rule refactoring independently rediscovers the predecessor construction or supplies a fresh proof-assistant verification of that bridge.

5 A concrete proof that becomes shorter

Put $F(n) := I(n, 0; u.Su)$ and $E(y) := (F(y) = y)$. The supplied new proof of $F(n) = n$ has the following structure.

1. IS proves $F(Sz) = S(F(z))$.
2. Under $E(z)$, transport along $F(z) = z$ in the template $F(Sz) = S(w)$ gives $F(Sz) = Sz$.
3. QP discharges that equation hypothesis and proves $Q_y(F(y) = y)$.
4. I0 gives $F(0) = 0$, and QE at n yields $F(n) = n$.

Counting the hypothesis, both computations, transport, QP, and QE gives six primitive nodes. The exact old translation of the QP node inserts the unused witness DP and DE construction, giving eight nodes.

Supplied named-syntax proof	Primitive nodes
Direct new proof of $I(n, 0; u.Su) = n$	6
Its translation into the old calculus	8
That old proof translated back to the new calculus	11
Separate old example with a nonvacuous existential witness	4
Its translation into the new calculus	7

The latter example deliberately gives the witness a free variable whose name matches the index binder in the matrix. Capture-avoiding substitution renames that binder and preserves the free parameter. The proof trees, their checked conclusions, and the translator are included in `audit/`.

This shows a useful reduction for a particular common induction pattern. It does not show that every new proof is shorter, that either supplied proof is shortest, or that the complete proof checker is smaller.

6 Implementation comparison and validation

6.1 The complete-source measurement

All three modules below are standalone. They use only Python's standard JSON reader and contain their own normalization, scope handling, inference dispatch, input entry point, and public verdict. They do not import a theorem table, a producer, the independent audit, or another checker.

Measurements were reproduced on CPython 3.12.14. The normalized text is obtained with `ast.unparse`, applied to the parsed source, followed by one newline. Significant token counts omit whitespace/layout, comments, and the terminal token; AST counts use `ast.walk`. Gzip uses compression level 9 and timestamp zero on the normalized bytes.

Complete module	Bytes	Tokens	AST	Gzip 9
Current `neutral_scope`	529	266	310	333
Literal-equation QP only	568	285	331	348
QP with derived compound Q-case	552	278	322	341

The literal-only candidate costs 39 additional normalized bytes (about 7.4%); the extended candidate costs 23 (about 4.3%). Neither improves any of the displayed source coordinates. The extra Q-root opening and its matrix validation offset the simpler induction interface in this arity encoding. This is an observed implementation result, not a lower bound proving that no encoding of the new basis can ever win.

6.2 Certificate interface

The compact interface follows the inherited scalar arity representation: equations are binary arrays, paired formulas unary arrays, and canonical closures triple repetitions of their matrix. Numerical zero, successor, and iteration share the corresponding scalar, unary, and ternary shapes in numerical positions. A depth-sensitive opener distinguishes numerical and formula positions, protects bound slots, and replaces the selected hole. The frozen arity-checker appendix specifies the original decoder, including its deliberately permissive aliases.

Input is a JSON array. Its final item is the closed goal; preceding four-field instruction packets are consumed backwards. An unused prefix is inert, as in the baseline. Acceptance requires exit status zero and standard output exactly `ACCEPT` followed by a newline; an exception rejects. Finite successful runs are the relevant certificates. Practical recursion and memory limits can reject otherwise valid, excessively large proofs.

The new canonical introduction packets are

Packet	Intended inference
`[[], 0, [], witness]`	D-case introduction
`[[], 0, 0, 0]`	Direct Q introduction

In the literal-only module the second packet requires an equational matrix. In the extended module it uses equation progress or compound Q-case according to the matrix root. The last packet field in a Q introduction is unused; it is retained to preserve the common four-field packet interface. This implementation detail must not be confused with an existential witness in the mathematical QP rule.

Nonempty major premises keep the inherited transport, QE, and DE protocol. Truthy initializer fields keep the inherited computation protocol. Its restricted direct computation supply is repaired using the baseline's proved zero representative; this package does not silently count a richer trusted arithmetic engine. The supplied process fixtures are already serialized, so their verification needs neither the upstream repository nor an external producer.

6.3 What was checked

The independent readable audit implements named terms, alpha equivalence, capture-avoiding substitution, all eight old/new rules, and both translations. It checks the displayed six-node proof, its translations, a nonvacuous witness example, and preservation under retained contexts. Negative cases exercise compound-progress rejection and eigenvariable capture or escape. These are checks of particular derivations, not proofs of a universal metatheorem.

The compact modules are also run as actual processes, in ordinary Python and with `-O`. Fixtures cover direct closure induction, contextual induction, paired introduction/elimination, nested closure formulas, equality transport under binders, inherited positive examples, and malformed or scope-leaking certificates. The extended candidate deliberately accepts admissible compound Q-case; that acceptance is not misreported as induction on a compound formula. All 290 process runs passed: 158 accepts, 132 rejects, and zero timeouts. Exact case inventories and per-process outcomes are recorded in `compact/evidence/validation.json` after reproduction.

A guard failure that matters. An unsuccessful Python Boolean guard is not automatically a rejecting certificate result. In this compact representation, returning `False` from a failed matrix test can put scalar zero into the context and close an altered scalar goal. The literal-only source therefore uses an invalid non-JSON complex sentinel for that failure. The extended source instead gives compound Q matrices the sound Q-case rule. This is why the shortest superficially plausible rewrite was not accepted as a size result.

Scope of assurance. The mathematical sequent-equivalence proof is explicit and independent of the programs. The Python audit provides supporting evidence and concrete proof objects. The raw compact interfaces reuse the inherited decoder and its scope arguments; neither the old nor the modified implementation was verified here in a proof assistant. The frozen upstream Lean files were inspected but not rebuilt in this environment. These distinctions prevent finite tests or existing Lean files from being presented as a new formal verification.

7 Why several more tempting deletions were not adopted

The repository already contains the important obstructions and reductions behind the following cautions. They are used to avoid repeating a failed proposal, not claimed as new results here.

Eliminating Q from the printed formula grammar. An unused existential coordinate makes $Q_y B(y)$ interderivable with $D_{x,y} B(y)$. The earlier mathematical presentation already uses only Eq/D objects together with reusable closure clauses. Removing the Q notation alone does not eliminate the conditional-universal resource or its implementation cost.

Making every atomic D introduction progress-only. That deletion loses a native theorem. The old calculus proves $D_{x,y}(y = 1)$ by its zero case: the base assumption is $0 = 1$, from which the required successor equation follows. A sound alternative interpretation of the reduced rules reads atomic D as $\exists x \forall y (E(x, y) \to E(x, Sy))$; it falsifies this example at $y = 1$. The current forced-progress obstruction gives the full argument [5]. Our refactoring keeps atomic D-case, so it avoids this loss.

Allowing Q-progress on arbitrary matrices. If the literal-equation restriction in QP is simply deleted, QP followed by QE gives induction for every native formula. Through the inherited logical macros this supplies unrestricted first-order induction. Such a change cannot be justified as the same iPRA presentation by the present argument. The extended rule uses the zero case for compound matrices instead.

Dropping numerical coverage in the ordinary realization. The coordinatewise structure $\mathbb{N}^2$ satisfies the iterator equations, separation, and contextual equation induction, but $(0, 1)$ is neither zero nor a componentwise successor. For equation induction, its solution set is $U_0 \times U_1$; the base supplies zero in both factors, and testing the step at $(r, 0)$ and $(0, r)$ makes each factor successor-closed. Hence both factors are all of $\mathbb{N}$. This illustrates why equation induction alone does not yield the arbitrary-formula case interface needed for full logic. The product obstruction is already developed in the supplied research family.

Hiding computation inside conversion. Replacing I0/IS by a contextual conversion rule still requires both rewrite patterns and an account of reflexivity. The baseline derives reflexivity from computation plus transport. An uncharged conversion engine would not be a smaller complete implementation.

8 Further research questions

1. **Can direct closure induction win under a different honest encoding?** The measured loss comes from distinguishing the two root openings. An alternative representation might share them without admitting compound induction or losing the exact Q/D round trips. A credible improvement must count the complete recognizer and exercise malformed, aliasing, and scope cases as well as canonical proofs.

2. **Can a simpler logical basis reduce an actual branch?** The present change removes a witness field, but keeps eight primitive patterns. A stronger result would delete a trusted inference case while preserving arbitrary native contexts and all logical formula composites. Restricting attention to closed equations would not settle that question.
3. **Can focused proof forms justify a simpler major-premise protocol?** Permuting logical introductions and eliminations might reduce general recursive major checking. Equation induction makes such a normalization theorem nontrivial. The theorem must precede any removal of proof checking, rather than be assumed by a producer.
4. **Can the current native bridge and the new refactoring be formalized together?** A direct Lean definition of the replacement derivation type, both derivation translations, and the native formula round trips would turn the present written equivalence into a mechanically checked result. The supplied frozen project is the natural comparison point.

The answer established here is therefore precise but limited: direct closure progress is an exact, simpler induction interface for the existing theory. It is not a new minimum-size complete axiomatization. The requested stronger improvement remains open in this package.

A Complete replacement checker

The following is the entire literal-equation-only module. Its size is 568 bytes after the stated normalization. Its arithmetic, scope, and JSON machinery is included in the measurement.

```
import json

def c(a, g=[[]], k=0):
    i = lambda a, t=0j, d=0: [i(b, t, d + 5 - len(a)) for b in a] if a * 0 == [] else a if k <= a < d else d %
      2 and i(t, d=3)
    if i(a) in g:
        return
    m, s, u, t = q()
    c(m, g, k)
    a = m in ([s, u], not s and [t] * 3) and i(t, u) == a and i(t, s) or a
    h = m * 3 or (u and [[s and [s], u, t], i(t, [s, u, t], 3) if s else u]) or (a := (i(*a, t, 2) if u == []
     else len(a[2]) == 2 and a[2] or 0j))
    k -= 1
    c(i(a, [k]), g + [i(h, (m or u == 0) and k)], k)
q = json.load(open(0)).pop
c(q())
print('ACCEPT')
```

The 552-byte extended module changes the introduction branch to permit derived Q-case for compound matrices. It is included separately as `compact/code/qprogress_extended.py`. The baseline complete source is `compact/code/baseline_529.py`.

B Reproduction and provenance

From the extracted package directory, run

```
python3 reproduce.py
pdflatex -halt-on-error report.tex
pdflatex -halt-on-error report.tex
```

The first command reruns the independent mathematical audit and the compact process tests, recomputes metrics, and writes `reproduction.json`. No network access or upstream checkout is needed for those Python checks. The PDF build uses standard LaTeX packages. Auxiliary build files are not needed to read the delivered PDF.

`CLAIMS.json` records both the positive claims and explicit nonclaims. `reference/provenance.json` identifies the fixed upstream commit and checksums of the copied mathematical and Lean sources. The unchanged upstream MIT-0 license is retained. The reference Lean project configuration is a convenience wrapper; its source modules are unmodified and a build requires Lean 4.32.0 with the matching mathlib. A new Lean proof of the refactoring is not supplied.

References

[1] V. Reshetnikov and ProveIt contributors. *Concise presentations of intuitionistic primitive recursive arithmetic*. ProveIt, pinned commit `8662bc0a00f4`, 2026. Pinned project and README.

[2] ProveIt contributors. *Equality and interderivability in the native scalar core.* `Lean/ConciseIPRA/NativeEq.lean`, theorem `NPrf.qIProg`, same pinned commit. Source.

[3] ProveIt contributors. T_{eq} *and iPRA are bi-interpretable.* `Lean/ConciseIPRA/BiInterpretation.lean`, definitions and theorems `biInterp`, `teq_roundTrip`, `pra_roundTrip`, `teq_consistent_iff`, same pinned commit. Source.

[4] ProveIt contributors. *The logical connectives as native formulas.* `Lean/ConciseIPRA/NativeLogic.lean` and `NativeGate.lean`, same pinned commit. Source.

[5] ProveIt research packages. *Why atomic case introduction cannot simply be removed.* `Research/research/theory/forced_progress_obstruction.tex`, same pinned commit. Source.

[6] S. Salehi. Decidable formulas of intuitionistic primitive recursive arithmetic. *Reports on Mathematical Logic* **36** (2002), 55–61. https://rml.tcs.uj.edu.pl/rml-36/36-SALEHI.pdf.